CS202 Design Pattern Report: Observer Pattern

Decoupling State Changes, Push vs. Pull Notifications, and C++ Memory Management

Group 52
Nguyễn Đình Minh Huy (Student ID: 25125083)
Trần Gia Huy (Student ID: 25125084)

August 27, 2026

Contents

1 Real-world Problem & Motivation

1.1 Background: Banking Event Notifications

In software development, maintaining consistency between core business entities and dependent logging or notification modules is a common requirement. In a simplified banking system, a `BankAccount` class manages account state, balance updates, and transaction records.

When a customer performs an operation (such as a deposit, withdrawal, or failed overdraft attempt), multiple subsystems need to respond:

1. **User Interface (UI):** Updates the displayed balance and transaction list.
2. **Email Service:** Sends transaction receipts and security alerts to the user.
3. **Audit Logger:** Writes timestamped audit records for compliance.
4. **Fraud Detection Engine:** Evaluates transaction parameters to identify suspicious activities.
5. **Mobile Push Service:** Sends alert notifications to the user's mobile device.
6. **Analytics Service:** Records transaction data for operational metrics.

1.2 Problem Statement

The objective is to allow the `BankAccount` class to notify multiple interested services when a transaction occurs without coupling the account logic to specific service implementations. Furthermore, services must be able to subscribe or unsubscribe at runtime without requiring changes to existing classes or recompilation of the core banking logic.

2 Naive Solution

In a basic implementation, the `BankAccount` class maintains direct member pointers to each concrete service. When a transaction takes place, `BankAccount` calls specific methods on each service sequentially.

```
namespace naive {

class UIUpdateService {
public:
    void updateUI(double amount, double balance) {
        std::cout << "[UI] Updated. Transaction: $" << amount << ", Balance: $" << balance << "\n";
    }
};

class EmailService {
public:
    void sendEmail(const std::string& accNum, double amount) {
        std::cout << "[Email] Sent receipt for account " << accNum << ". Amount: $" << amount << "\n";
    }
};

class LoggerService {
public:
    void logTransaction(const std::string& accNum, double amount) {
        std::cout << "[Logger] Logged transaction for " << accNum << ". Amount: $" << amount << "\n";
    }
};

class FraudDetectionService {
public:
    void detectFraud(const std::string& accNum, double amount) {
        std::cout << "[Fraud AI] Checked account " << accNum << " for $" << amount << ". Status: OK.\n";
    }
};

class MobileAppService {
public:
    void pushNotification(double amount) {
        std::cout << "[Mobile App] Push alert sent. Amount: $" << amount << "\n";
    }
};

class AnalyticsService {
public:
    void trackEvent(const std::string& eventType, double amount) {
        std::cout << "[Analytics] Event '" << eventType << "' logged for $" << amount << "\n";
    }
};

// BankAccount directly coupled to concrete service classes
class BankAccount {
```

```
private:
    double balance;
    std::string accountNumber;

    // Direct dependencies on 6 concrete classes
    UIUpdateService* uiService;
    EmailService* emailService;
    LoggerService* loggerService;
    FraudDetectionService* fraudService;
    MobileAppService* mobileService;
    AnalyticsService* analyticsService;

public:
    BankAccount(std::string accNum,
                UIUpdateService* ui, EmailService* email,
                LoggerService* logger, FraudDetectionService* fraud,
                MobileAppService* mobile, AnalyticsService* analytics)
        : balance(0.0), accountNumber(accNum),
          uiService(ui), emailService(email), loggerService(logger),
          fraudService(fraud), mobileService(mobile), analyticsService(
    analytics) {}

    void deposit(double amount) {
        balance += amount;

        // Explicit dispatch calling each service method directly
        uiService->updateUI(amount, balance);
        emailService->sendEmail(accountNumber, amount);
        loggerService->logTransaction(accountNumber, amount);
        fraudService->detectFraud(accountNumber, amount);
        mobileService->pushNotification(amount);
        analyticsService->trackEvent("deposit", amount);
    }
};

} // namespace naive
```

Listing 1: Naive Bank Account with Direct Service Coupling (`naive.cpp`)

2.1 Step-by-Step Execution of the Naive Solution

To illustrate the behavior of the naive implementation, consider depositing $500.00:

1. **Initialization:** The client instantiates all 6 concrete service objects and passes their pointers to the `BankAccount` constructor.
2. **State Update:** Invoking `bankAccount.deposit(500.0)` increments the internal `balance` attribute by $500.00.
3. **Sequential Dispatch:** The `deposit()` method executes the following calls:
    - `uiService->updateUI(500.0, 500.0)`
    - `emailService->sendEmail("ACC-101", 500.0)`
    - `loggerService->logTransaction("ACC-101", 500.0)`
    - `fraudService->detectFraud("ACC-101", 500.0)`

- `mobileService->pushNotification(500.0)`
- `analyticsService->trackEvent("deposit", 500.0)`

3 Analysis of Naive Solution Drawbacks

The naive approach presents several design limitations:

Issue	Cause in Code	Effect on System
Open-Closed Principle (OCP) Violation	Adding a service requires modifying the `BankAccount` member variables, constructor, and `deposit()` method.	Existing banking code must be altered and retested whenever notification requirements change.
Single Responsibility Principle (SRP) Violation	`BankAccount` manages balance state while also coordinating logging, UI updates, and email dispatching.	Increases class complexity and couples financial logic to external I/O operations.
Tight Coupling	`BankAccount` references concrete classes and depends on their individual method signatures.	Signature changes in service classes require corresponding edits in `BankAccount`.
Static Subscriptions	Service pointers are assigned at construction time.	Services cannot be enabled, disabled, or substituted dynamically during execution.
Testing Overhead	Testing `BankAccount` requires instantiating or managing instances of all 6 concrete services.	Increases testing dependencies and makes isolated unit testing more difficult.

Table 1: Design Issues in the Naive Implementation

4 Introduction to the Observer Design Pattern

4.1 Definition and Intent

According to the Gang of Four (GoF), the **Observer Pattern** is defined as:

> *"Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically."*

The pattern separates the object that generates events (the **Subject**) from the objects that handle those events (the **Observers**) by using an abstract interface.

4.2 Key Structural Components

- **Subject (`BankAccount`):** Maintains a collection of observer pointers and provides subscription methods:
  - `attach(observer)`: Registers a new subscriber.

  - `detach(observer)`: Removes a registered subscriber.
  - `notify(type, amount)`: Iterates through subscribers and calls their update method.

- **Observer Interface (`AccountObserver`):** Declares the abstract update method:

```
virtual void onTransaction(const std::string& type, double amount,
    double balance) = 0;

```

- **Concrete Observers (`UIObserver`, `EmailObserver`, `LoggerObserver`, etc.):** Implement the interface to execute their specific response logic.

- **Push vs. Pull Notification Models:**
  - **Push Model:** The Subject sends state details (`type`, `amount`, `balance`) directly as method arguments during the notification call.
  - **Pull Model:** The Subject passes a reference to itself, and observers query the specific state attributes they need.

5 UML Class Diagrams

5.1 General GoF Observer Pattern Structure

The general Observer pattern structure connects the `Subject` to the abstract `Observer` interface rather than concrete subscriber classes.

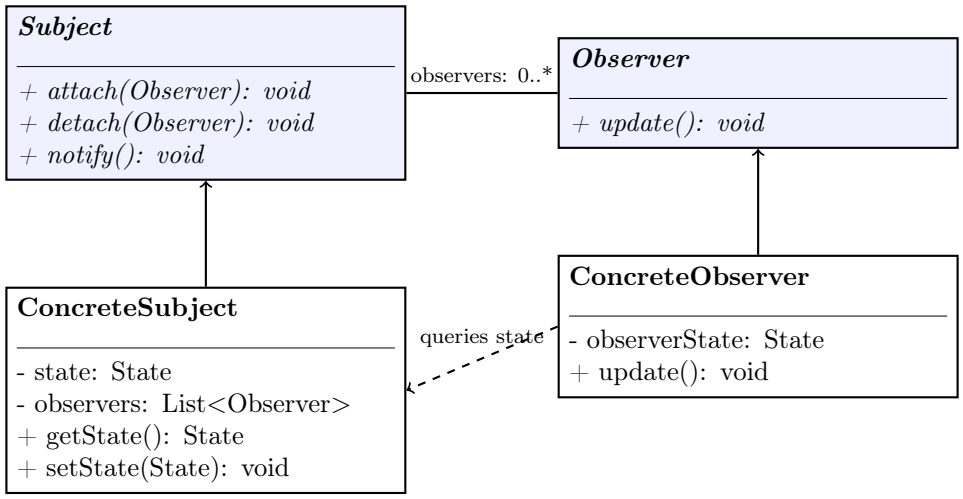

Figure 1: General GoF Observer Pattern UML Structure

5.2 Bank Account Notification Class Diagram

In the bank account system, `BankAccount` acts as the Subject, storing a collection of `std::shared_ptr<AccountObserver>` instances. All 6 concrete notification classes implement the `AccountObserver` interface.

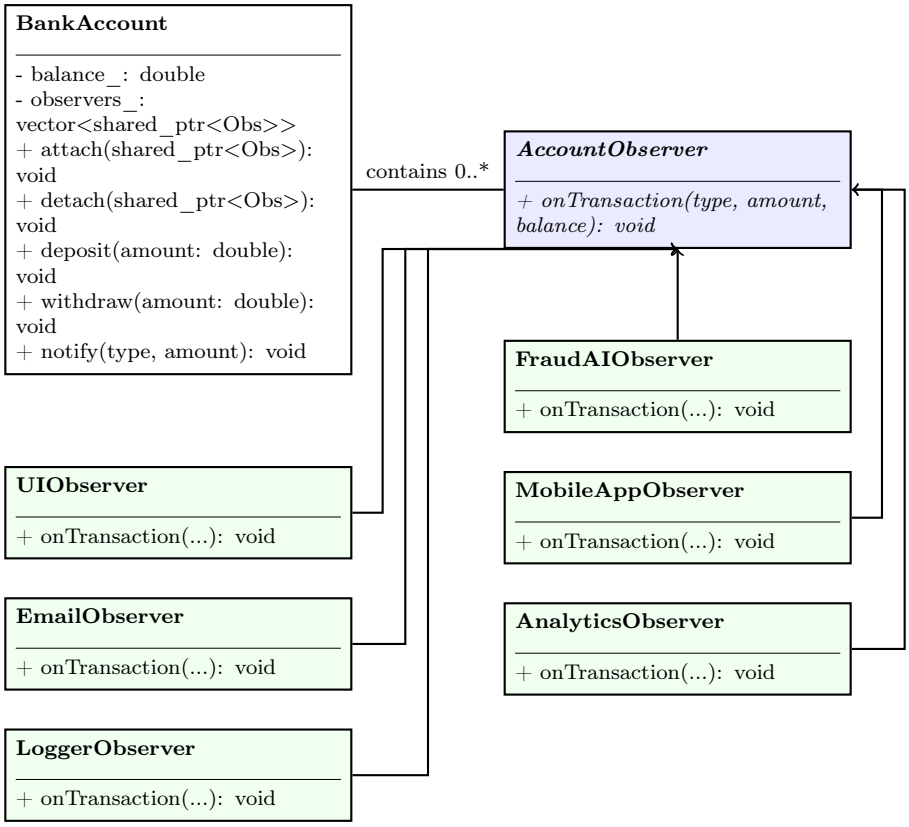

Figure 2: Bank Account Observer Pattern Class Hierarchy

6 Pattern-Based Implementation in C++17

6.1 Memory Management Considerations

In C++, managing observer lifetimes requires addressing potential reference issues:

- **Raw Pointers:** If an observer is deleted while remaining in the subject's list, subsequent notifications can result in dangling pointer access.
- **Smart Pointers (`std::shared_ptr` and `std::weak_ptr`):**
  - `std::shared_ptr`: Keeps the observer instance valid while it is held in the subject's subscription list.
  - `std::weak_ptr`: Allows the subject to observe whether the subscriber still exists via `lock()` before invoking callbacks, preventing memory leaks when observers are destroyed externally.

6.2 Source Code Implementation

```
#include <iostream>
#include <memory>
#include <vector>
#include <string>
#include <algorithm>

// Observer Interface
```

```cpp
class AccountObserver {
public:
    virtual ~AccountObserver() = default;
    virtual void onTransaction(const std::string& type, double amount,
                               double currentBalance) = 0;
};

// Subject Class
class BankAccount {
public:
    BankAccount(double initialBalance = 0.0) : balance_(initialBalance)
    {}

    void attach(std::shared_ptr<AccountObserver> observer) {
        observers_.push_back(observer);
    }

    void detach(std::shared_ptr<AccountObserver> observer) {
        observers_.erase(
            std::remove(observers_.begin(), observers_.end(), observer)
    ,
            observers_.end()
        );
    }

    void notify(const std::string& type, double amount) {
        for (const auto& observer : observers_) {
            observer->onTransaction(type, amount, balance_);
        }
    }

    void deposit(double amount) {
        balance_ += amount;
        std::cout << "[BankAccount] Deposited $" << amount << ". New
    balance: $" << balance_ << "\n";
        notify("Deposit", amount);
    }

    void withdraw(double amount) {
        if (amount > balance_) {
            std::cout << "[BankAccount] Rejected: Insufficient funds
    for withdrawal of $" << amount << "\n";
            notify("FailedWithdrawal", amount);
        } else {
            balance_ -= amount;
            std::cout << "[BankAccount] Withdrew $" << amount << ". New
     balance: $" << balance_ << "\n";
            notify("Withdrawal", amount);
        }
    }

    double getBalance() const { return balance_; }

private:
    double balance_;
    std::vector<std::shared_ptr<AccountObserver>> observers_;
```

```
59 };
```

Listing 2: Observer Interface and BankAccount Subject Implementation (`pattern.cpp`)

```
class UIObserver : public AccountObserver {
public:
    void onTransaction(const std::string& type, double amount, double currentBalance) override {
        (void)type; (void)amount;
        std::cout << "[UI Observer] Balance display updated to $" << currentBalance << "\n";
    }
};

class EmailObserver : public AccountObserver {
public:
    void onTransaction(const std::string& type, double amount, double currentBalance) override {
        (void)currentBalance;
        if (type == "Deposit") {
            std::cout << "[Email Observer] Alert sent: A deposit of $" << amount << " was made.\n";
        } else if (type == "Withdrawal") {
            std::cout << "[Email Observer] Alert sent: A withdrawal of $" << amount << " was made.\n";
        } else if (type == "FailedWithdrawal") {
            std::cout << "[Email Observer] Security Alert: Withdrawal of $" << amount << " failed (Insufficient funds).\n";
        }
    }
};

class LoggerObserver : public AccountObserver {
public:
    void onTransaction(const std::string& type, double amount, double currentBalance) override {
        std::cout << "[Logger Observer] Transaction logged: " << type
                  << " of $" << amount << ", new balance: $" << currentBalance << "\n";
    }
};

class FraudAIObserver : public AccountObserver {
public:
    void onTransaction(const std::string& type, double amount, double currentBalance) override {
        (void)currentBalance;
        if (amount > 10000.0) {
            std::cout << "[Fraud AI] WARNING: Suspicious activity detected: " << type << " of $" << amount << "\n";
        } else {
            std::cout << "[Fraud AI] Scan complete: " << type << " of $" << amount << " marked OK.\n";
        }
    }
};

class MobileAppObserver : public AccountObserver {
public:
    void onTransaction(const std::string& type, double amount, double currentBalance) override {
        (void)currentBalance;
```

```
47          std::cout << "[Mobile App] Push notification: " << type << " of
        $" << amount << " processed.\n";
48      }
49  };
50
51  class AnalyticsObserver : public AccountObserver {
52  public:
53      void onTransaction(const std::string& type, double amount, double
        currentBalance) override {
54          (void)currentBalance;
55          std::cout << "[Analytics Observer] Event registered: Type=" <<
        type << ", Volume=$" << amount << "\n";
56      }
57  };
```

Listing 3: Concrete Observer Implementations

6.3 Execution Walkthrough & Dynamic Subscription Demo

The execution driver demonstrates attaching all 6 observers, running a deposit, detaching the Email and Mobile App observers at runtime, and verifying the subsequent withdrawal output.

```
void runPatternDemo() {
    BankAccount account(1000.0);

    auto ui = std::make_shared<UIObserver>();
    auto email = std::make_shared<EmailObserver>();
    auto logger = std::make_shared<LoggerObserver>();
    auto fraud = std::make_shared<FraudAIObserver>();
    auto mobile = std::make_shared<MobileAppObserver>();
    auto analytics = std::make_shared<AnalyticsObserver>();

    // 1. Attach all 6 observers
    account.attach(ui); account.attach(email); account.attach(logger);
    account.attach(fraud); account.attach(mobile); account.attach(
    analytics);

    // 2. Perform Deposit
    std::cout << "\n--- TEST 1: Deposit $500.00 (All 6 Observers Active
    ) ---\n";
    account.deposit(500.0);

    // 3. Dynamic Detachment
    std::cout << "\n--- TEST 2: Detaching Email & Mobile Observers ---\
    n";
    account.detach(email);
    account.detach(mobile);

    // 4. Perform Subsequent Withdrawal
    std::cout << "\n--- TEST 3: Withdrawal $200.00 (Email & Mobile
    Inactive) ---\n";
    account.withdraw(200.0);
}
```

Listing 4: Observer Pattern Execution Demo (`runPatternDemo`)

Table 2 summarizes the execution trace and notification delivery across the dynamic lifecycle:

Step	Operation	Active Observers	Dispatched Output
1	Init (`Bal=$1000`)	UI, Email, Logger, Fraud, Mobile, Analytics	Initial state initialized with 6 subscribers attached.
2	`deposit(500.0)`	All 6 Observers	UI updates balance to $1500; Email sends receipt; Logger records entry; Fraud AI checks amount; Mobile sends alert; Analytics logs event.
3	`detach(email, mobile)`	UI, Logger, Fraud, Analytics	Email and Mobile App observers removed from list.
4	`withdraw(200.0)`	UI, Logger, Fraud, Analytics	UI updates balance to $1300; Logger records $200 withdrawal; Fraud AI checks amount; Analytics logs event. Email and Mobile receive no events.

Table 2: Dynamic Subscription and Event Broadcast Trace

7 Evaluation of Trade-offs

7.1 Advantages

- **Open-Closed Principle (OCP):** New observer classes (such as tax calculators or external reporting tools) can be added without modifying `BankAccount`.
- **Loose Coupling:** `BankAccount` depends only on the abstract `AccountObserver` interface, separating transaction logic from output mechanisms.
- **Runtime Configuration:** Observers can attach or detach dynamically based on application requirements.
- **Broadcast Distribution:** A single state update automatically notifies all registered observers.

7.2 Disadvantages and Limitations

- **Lapsed Listener Issues:** If observers are destroyed without detaching, the subject may retain inactive references, causing resource leaks or undefined behavior when raw pointers are used.
- **Unordered Notification:** Observers are invoked in list order, so individual observers must not assume specific execution ordering relative to other observers.
- **Cascading Updates:** If an observer calls a method on the subject that triggers another notification, it can lead to unintended recursive updates.

8 Applications

The Observer pattern is widely applied in software engineering whenever loose coupling between state providers and dependent consumers is required:

- **Graphical User Interface (GUI) Event Systems:** Modern UI toolkits use the Observer pattern to handle user interactions. UI components (such as buttons, sliders, and text fields) serve as event sources, while event handlers register as observers to process clicks, keystrokes, and focus changes.
- **Model-View-Controller (MVC) Architectures:** In MVC (and MVVM architectures), the Model acts as the Subject maintaining application state. Multiple Views register as observers with the Model to automatically update and re-render the display whenever the underlying data changes.
- **Publish-Subscribe and Message-Driven Systems:** Message brokers and event buses implement broadcast distribution where message topics act as subjects and distributed microservices subscribe to receive event payloads asynchronously.
- **State Management and Reactive Data Streams:** Modern state containers and reactive streams allow UI components or backend workers to subscribe to state stores, triggering downstream updates automatically when state changes occur.
- **System Monitoring and Logging Services:** Telemetry pipelines, audit loggers, and health check services attach to critical business processes as observers to record events and detect anomalies without altering core transactional logic.

9 Interpreter and Mediator Integration

Software architectures rarely utilize design patterns in isolation. The Observer pattern combines with the Interpreter and Mediator patterns to establish decoupled, event-driven, and reactive application pipelines.

9.1 Combining Observer with the Interpreter Pattern

In standard evaluation pipelines, expression trees are evaluated on-demand when explicitly triggered by a caller. Integrating the Observer pattern transforms this model into a push-based reactive rule engine.

In this architecture, the evaluation symbol table (`Context`) functions as an observable Subject. The root nodes of domain-specific Abstract Syntax Trees (ASTs) register as observers listening for updates to specific variable identifiers. Whenever a variable's value changes within the Context, the Context notifies all dependent expression trees. The trees re-evaluate their logical predicates automatically and trigger downstream actions without requiring polling loops or manual re-invocation.

9.2 Combining Observer with the Mediator Pattern

In complex systems with multiple interacting components, direct peer-to-peer subscriptions can lead to intricate point-to-point connections. Combining the Observer and Mediator patterns resolves this issue by introducing a centralized Event Aggregator or Event Bus.

Under this model, components do not subscribe directly to each other. Instead, they register as observers with a central `EventMediator`. When an event occurs, the publishing component sends the event details to the Mediator. The Mediator inspects the event payload, applies routing or filtering policies, and dispatches notifications to the relevant subscriber components. This approach simplifies the communication topology from an $O(N^2)$ mesh to an $O(N)$ star topology while maintaining loose coupling.

10 Conclusion

The Observer Design Pattern provides a structured way to decouple state-holding subjects from dependent notification and logging modules. By using an abstract interface and maintaining dynamic subscriber collections, the pattern supports the Open-Closed and Single Responsibility Principles.

In C++17, utilizing smart pointers (`std::shared_ptr` and `std::weak_ptr`) ensures safe object lifetime management and prevents dangling references. When integrated with the Interpreter and Mediator patterns, the Observer pattern supports reactive rule engines and centralized event-driven architectures.

References

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Meyers, S. (2014). *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- Refactoring.Guru. *Design Patterns: Observer Pattern*. `https://refactoring.guru/design-patterns/observer`